Architecture for the Local Operation of Intelligence

A Pattern Language for Distributable AI Operating Doctrine

Authors: David Nguyen (primary architect); DRACO 67, DRACO 68, DRACO 69 (deployment authors). Contributing assets: POLARIS (intelligence operations and source verification), MOBIUS 9 (overwatch), doc-coauthoring skill.

Date: 2026-05-05 **Status:** Focused white paper edition. Long-form record preserved separately.

Abstract

Most uses of artificial intelligence today are containers. A user asks a question. An AI assistant returns an answer. The container empties when the conversation closes. This paper proposes a different posture: a framework for building, configuring, and distributing durable AI operating doctrine at the individual operator level.

The framework is structured in three layers: bones (universal mechanisms shared by every install), flavor (operator-specific configuration), and presence (configurable depth of relational tone). It runs on a single ethical commitment, system-pointed self-audit rather than user-pointed observation, and a distribution mechanism that lets the architecture travel to other operators with their own contexts. The contribution is design research from one operator's worked deployment; the methodology and limitations are stated explicitly.

The paper draws on contemporary AI literature on cognitive architectures and persistent memory (Packer et al., 2023), agentic orchestration standards (Anthropic, 2024b), user-level alignment (Bai et al., 2022), and from older traditions: Christopher Alexander's pattern languages (Alexander et al., 1977), Donald Schön's reflective practitioner work (Schön, 1983), David Parnas's information hiding (Parnas, 1972), and the ethical foundations articulated by Helen Nissenbaum (2009) and Shoshana Zuboff (2019).

The framework's contribution is the architecture itself: an articulated, distributable system that other operators can install, configure, and inhabit on their own terms.

1. The Problem and the Framing

1.1 The Cup Problem

Most uses of AI today follow the same shape. A user asks a question. The system answers. The conversation ends, and the next conversation starts from nothing. The intelligence behaves like water: it takes the shape of whatever container holds it. Most users are working with cups.

This is the cup-style paradigm of human-AI interaction. The vendor supplies the container by default. The user adjusts lightly through system prompts, custom instructions, or project-level files. Everything inside the container is single-use. The intelligence does not persist across sessions. The user's accumulated context does not persist. The operating habits that should govern interaction do not persist beyond the local session boundary. The next session begins from the same baseline state, regardless of what was learned in the previous one.

Modern vendor offerings have begun to ship persistence at their layer: ChatGPT memory features, Anthropic Project containers, custom instructions across major chat interfaces. The pure cup with no persistence is increasingly a historical artifact. But vendor-side persistence travels only as far as the vendor's product travels. An operator who configures memory in one vendor's interface and switches vendors must reconfigure from scratch. The operator does not own the architecture; the vendor does.

What this paper proposes is different. It is a framework for building, configuring, and distributing durable AI operating doctrine at the individual operator level: an architecture for the local operation of intelligence. The framework moves the operator one level above the vendor's persistence layer, into operator-owned architecture composed of standardized parts that travel across vendors and across model upgrades. The operator becomes the architect.

1.2 Origin: Why the Architecture Was Needed

The framework's origin is a specific observation made during sustained advanced use of AI on demanding tasks: academic writing, code production, multi-file projects, long-horizon research. The operator imposed tight requirements: specific output formats, rigorous citation discipline, structured planning before execution, refusal of fabrication, persistence of context across sessions through manual artifact management. Under those requirements, the AI exhibited a reproducible failure mode. When the AI hit operational resistance, its easiest path was to defer the work back to the operator rather than find a better solution itself.

The canonical instance was file access. When the AI could not retrieve a needed file, the response pattern was a request that the operator download the file manually and resubmit it through a different channel. The request worked. The file would be passed back. The work would resume. The pattern was efficient in a narrow sense (the immediate task completed) and inefficient in a broader sense (the operator was performing labor the AI was supposed to perform). More consequentially, the pattern generalized. File access was the visible instance; the deeper behavior was the AI's tendency, when blocked, to take the lowest-friction path forward, which was almost always asking the operator to do something. Self-reflection, deploying a different sub-agent, researching proven approaches, or honestly declaring an uncertainty boundary that the operator could decide on were higher-friction paths. The architecture had no mechanism that pushed the AI toward those higher-friction options.

This observation is the framework's origin. The framework is not, in the first instance, a contribution to AI capability research. It is a contribution to how the operator and the AI organize their work together so that the AI's path of least resistance does not default to "ask the operator to do it."

1.3 Cup and City

The contrast that anchors the framework is between cup-style use and city-style use.

A cup is a single-purpose container. It holds water for one drink. It does nothing else. The intelligence that flows into a cup-style AI session resolves a single problem and dissipates when the session closes.

A city is something else entirely. A city is a persistent built environment composed of many specialized structures: districts, streets, libraries, courts, infrastructure. Water in a city flows through aqueducts to fountains, through plumbing to households, through cisterns into reservoirs. The water is not the city. The water is the substance that the city's architecture organizes for living. The city makes possible kinds of life that no cup can support.

The cup-versus-city distinction maps onto contemporary AI use. A cup-style operator asks discrete questions and receives discrete answers. A city-style operator inhabits a structured architecture: a Gate that classifies incoming requests, a Protocol that runs through every meaningful task, a Logbook that records what worked and what slipped, a Memory that holds context across sessions, a Pitfall Watch that surfaces recurring patterns. The intelligence flows through all of it. The architecture persists. The operator inhabits the city over time.

The conceptual prerequisite for the framework, therefore, is the recognition that a city is possible. An operator who has only ever used cups cannot read blueprints for a city, because the building has no place in their imagination yet. This paper accepts that prerequisite as a problem to be solved at the front of the framework, not assumed away.

1.4 Roman Aqueducts: The Conceptual Anchor

The closest historical analog for what the framework attempts is Roman water infrastructure. The Romans built aqueducts that delivered water across distances of tens or hundreds of kilometers, using gravity flow, sealed channels, and engineered durability. Frontinus, the curator of Rome's aqueducts in the late first century CE, documented the system as an administrative and engineering achievement (Frontinus, 1973/c. 98 CE). The system worked because the engineering was conservative, the standards were enforced across centuries of construction, and the application was deliberately agnostic. Frontinus did not mandate what the water was used for. The aqueduct delivered water reliably to public fountains, to baths, to private cisterns, to industrial uses. The aqueduct did not moralize about its application. It moved the substance and let the city decide (Hodge, 2002).

This is the Roman aqueduct frame applied to AI architecture: build infrastructure that delivers intelligence reliably to operators, and let the operators decide what to do with it. The architecture's success is measured by its durability and consistency, not by the specific applications operators construct. The framework remains agnostic about whether an operator uses it for academic research, professional practice, family operations, or recreational projects. The water arrives. The operator decides.

A modern technical analog reinforces the point. The Internet's TCP/IP stack functions as an Hourglass Architecture: a thin, durable waist that defines the protocol for transporting data, with an open layer of applications above and an open layer of physical media below (Akhshabi & Dovrolis, 2011). TCP/IP carries email, web browsing, video streaming, voice calls, and uses unanticipated by its original designers. The framework adopts the same architectural posture at a different layer of the stack: standardized infrastructure for AI operating doctrine that does not dictate what the operator builds with it.

1.5 The Monument Frame

The fourth conceptual foundation addresses what success looks like for the framework once it has been distributed. A monument is a durable structure intended to persist long after its builders are gone. Some who encounter the monument over time will be inspired by it. Some will pass it without notice. Some will deface it. Some will use the monument's foundations to build something the architects did not anticipate. The monument's standing is not validated by any single interaction. The monument earns its standing by being well-built and persisting across time.

This frame matters because the framework's distribution is selective and the outcomes are uncontrollable. When the package travels to a new operator, the recipient may use it deeply, may use it shallowly, may abandon it, or may apply it in ways the architects could not anticipate. The framework's success is not measured by whether each recipient inhabits a fully developed city. It is measured by whether the architecture is sound enough to support whatever the recipient chooses to build, including the case where the recipient builds nothing at all.

The monument frame moves the validating variable from interaction quality to architecture quality. This is a deliberate methodological choice that aligns with the framework's neutrality principle, developed in detail later in the paper.

1.6 Methodology and Scope

This paper presents a framework with one worked operator deployment as case study. It is design research, not empirical validation across multiple deployments. The framework emerged from iterative practice with one operator over several weeks, with the AI system itself serving as a thinking partner during the doctrine's articulation. Each component was tested, corrected, and refined through actual use, with corrections persisted as rule additions to the operating files.

This methodology has clear precedent. Christopher Alexander's pattern language emerged from observation and iterative refinement rather than from controlled experimental study (Alexander et al., 1977). Donald Schön's reflective practitioner concept was theorized from observation across many domains rather than from quantitative measurement (Schön, 1983). Foundational HCI research, including Eric Horvitz's mixed-initiative principles, was articulated as design principles supported by examples and theoretical argument (Horvitz, 1999). The methodological tradition is well established for framework contributions of this kind.

The paper makes no claim of empirical validation across multiple operators. The framework's transferability is the next research question, addressed only briefly here as future work. What this paper contributes is the framework itself: an articulated, distributable architecture that other operators can test against their own contexts.

1.7 What the Paper Does

The paper makes three primary contributions.

First, it articulates a three-layer architecture (bones, flavor, presence) for distributable AI operating doctrine at the individual operator level. The architecture is scale-neutral. It supports operators who need a small forward operating base, operators who build mid-scale communes, and operators who construct fully developed cities.

Second, it presents a doctrine of reflexive operation: structured self-audit pointed at the system's own operation rather than at the user. This is the ethical core of the framework. Pattern identification pointed at the user is surveillance. Pattern identification pointed at the system is consistency maintenance. The framework adopts the latter and treats the distinction as load-bearing.

Third, it proposes a distribution mechanism that lets the framework travel to other operators with their own contexts and threat models. The mechanism includes recipient profiles, a cargo manifest, a FIRST BOOT installation step, and a neutrality principle that prevents the framework from dictating use across distinct deployment contexts.

The framework is one specific instance of a class of possible architectures for the local operation of intelligence. Other instances in the class exist or could exist, optimized for different operator profiles, different domains, different cultural contexts. This paper does not claim to be the canonical or unique solution. It claims to be one well-articulated instance, contributed in a form other operators can install and configure on their own terms.

The paper proceeds as follows. Section 2 develops the three-layer architecture in detail. Section 3 specifies the reflexive operation doctrine. Section 4 presents the distribution mechanism. Section 5 addresses persistence, limitations, and future research, closing with the monument frame.

Throughout, the framework treats the operator as the architect of their own city. The framework supplies the bones, the patterns, the protocol, and the doctrine. The operator brings the water.

2. The Three-Layer Architecture

2.1 Overview

The framework's architectural contribution is a three-layer model: bones, flavor, presence. The terms are deliberately concrete. They map onto the operator's experience of organizing intelligence in their life: the structural pieces that hold the system together, the personal configuration that makes the system theirs, and the modulated presence the system maintains in the operator's environment.

Each layer is operationally complete on its own. A package shipped at the bones-only level produces a system that runs, classifies, follows protocol, maintains a logbook, and learns from corrections. A package configured to the flavor level adds operator-specific identity and personalization. A package extended to the presence level adds the depth options that turn a useful tool into a sustained working partner. No layer requires the layer above it.

The three-layer split echoes the architectural separation principle that David Parnas formalized in his 1972 paper on system decomposition: modules should be defined by the design decisions they hide, not by the surface order in which an operator might encounter them (Parnas, 1972). The bones hide the universal mechanism. The flavor hides the operator-specific declarations. The presence hides the relational configuration. An operator can change the flavor without disturbing the bones. An operator can choose to leave the presence layer empty without breaking the system. This is information hiding applied to AI operating doctrine.

2.2 Bones: Universal Mechanisms

The bones layer is the architecture's load-bearing structure. It ships with every package and operates independently of any operator-specific declarations. The bones contain the mechanisms by which intelligence is classified, governed, recorded, and improved. Five primitives compose the bones layer.

The Gate. Every incoming task passes through a classification step before any other action. The Gate distinguishes simple tasks (single-step, reversible, unambiguous, low stakes) from complicated tasks (multi-step, ambiguous, irreversible, or touching consequential domains). Simple tasks proceed to direct execution. Complicated tasks invoke the Protocol. When the classification is genuinely ambiguous, the Gate prompts the operator to disambiguate rather than guess. The Gate is the first piece of bones because it determines which other mechanisms apply. Without classification, the architecture cannot decide how much process to run. The Gate is the framework's analog to the triage step in medical or military operations: the brief assessment that determines the rest of the response.

The Protocol. For tasks classified as complicated, the framework runs an explicit five-step protocol: Clarify, Plan, Approve, Execute, Report. Clarify surfaces the questions whose answers would change the output. Plan articulates the goal, constraints, definition of done, and assets needed. Approve pauses for the operator's explicit go-ahead. Execute runs the plan with structured status updates. Report closes the task with a one-line summary of result, output paths, and any unresolved items. The Protocol is the architecture's primary work-quality mechanism. Tasks that follow the Protocol arrive at the operator clean. Tasks that bypass the Protocol drift into ambiguity, scope creep, or surprise output. The five-step structure has antecedents in mixed-initiative interaction research (Horvitz, 1999), where the principle that an agent should articulate intent before acting and should yield initiative back to the user at appropriate moments is foundational.

The NEVER List as a Mechanism. The framework distinguishes carefully between the NEVER list as a mechanism (which is bones) and the specific contents of any NEVER list (which is flavor). The mechanism is the structural fact that hard constraints exist, that they are immutable within the operator's deployment, that they are read on every session start, that they cannot be overridden by reflexive operation, and that they sit in a sovereign position above the rest of the doctrine. The contents are whatever the operator declares. Anthropic's Constitutional AI work establishes the technical and philosophical basis for treating a written set of principles as load-bearing in operational behavior (Bai et al., 2022). The original Constitutional AI applied the principle at the model level, for safety alignment. The framework here applies the same principle at the operator level: the operator's declared NEVERs function as a personal constitution that the system enforces during operation.

The Logbook. The framework records its own operation. The Logbook is the persistent ledger where the system tracks missions completed, status updates sent, corrections received and absorbed, hard-constraint violations with date and fix, achievements unlocked, and recurring patterns observed. The Logbook serves three architectural functions. First, it provides honest accounting: the operator can audit what actually happened across sessions, not what either party remembers. Second, it provides continuity across instances: a new instance loading the doctrine reads the Logbook and inherits the operational history. Third, it provides the substrate for pattern detection that lets the architecture identify drift, celebrate consistency, and surface concerns reflexively. The Logbook concept echoes institutional reflective practice in skilled work: medical morbidity-and-mortality conferences (Pierluissi et al., 2003), military after-action reviews (Headquarters, Department of the Army, 1993).

The Adaptive Learning Loop. When the operator corrects the system, the framework absorbs the correction as a rule rather than as a one-off behavior change. The operator corrects. The system identifies which package file the correction belongs in. The system proposes the rule update with explicit phrasing. The operator approves or modifies. The rule lands in the package. The next session inherits the rule. The architectural significance is that corrections do not vanish. In a cup-style system, a correction in one session has no purchase on the next. In the framework, the correction becomes durable doctrine. The system's behavior improves as the operator's preferences accumulate, but the improvement is operator-controlled, explicit, and auditable. This avoids the recurring problem in personalized systems where the system absorbs preferences without operator awareness, drifting toward behavior the operator never explicitly endorsed.

The bones layer collectively functions as a pattern language for AI operating doctrine. Christopher Alexander, Sara Ishikawa, and Murray Silverstein established the methodology of distributable architectural knowledge in pattern form (Alexander et al., 1977). The methodology was adopted in software engineering (Gamma et al., 1994) and is now applied here to AI operating doctrine for individual operators. The five primitives are designed to compose with one another and with the operator's specific configuration.

2.3 Flavor: Operator-Specific Configuration

The flavor layer holds everything that makes the architecture the operator's. The bones run the same way for every operator. The flavor is what tells the system who it serves, in what voice, with what hard constraints, in what life context. Flavor is configuration data, not mechanism. Every flavor entry has a default behavior at the bones level (typically a null or generic value) and an explicit slot the operator fills.

The flavor begins with **identity**. The operator names the system, declares a callsign convention if desired, and specifies voice preferences. These declarations shape every output. They are the most visible layer of the operator's configuration and the easiest to articulate. The act of naming is small but architecturally significant. It marks the transition from cup to city.

The flavor populates the **NEVER list mechanism with operator-specific content**. The operator declares what the system must never do regardless of other instructions: domains the system must not touch, communications that must always be drafts and never sent, files that cannot be deleted without explicit permission, facts that must not be invented. The architectural commitment is that the NEVER list, once declared, sits sovereign. Reflexive operation cannot modify it. Only an explicit operator amendment can change a NEVER. This sovereignty is the architecture's primary safeguard against optimization drift.

The flavor declares **ruin domains**: the categories in the operator's life where errors compound rather than recover. For a student, ruin domains might include academic standing and financial aid eligibility. For a professional, professional licensing and reputation. For a beneficiary of an institutional program, the records and entitlements that program governs. Tasks that touch a ruin domain receive heightened scrutiny: the Protocol runs more carefully, status updates surface earlier, sub-agent deployment is more conservative, and any decision that risks the domain triggers explicit operator confirmation.

The flavor includes **named advisors and relationships**: doctors, therapists, mentors, professional advisors, family members, close colleagues, dependents. The system uses these to handle

correspondence, scheduling, and any task where the relationship context shapes appropriate action. The flavor populates these contexts; the bones enforce the protections.

The flavor specifies the **connector loadout**: the tools available in the operator's environment. Different operators have different loadouts. An academic might have library databases and citation managers. A clinician might have EHR access. A developer might have version control and CI/CD systems. The framework treats the loadout as an operator declaration that shapes what the architecture can do.

The flavor culminates in a **memory file**: the operator's personal context that the system reads on every session start. The file holds whatever the operator has declared as durably relevant. It is the operator's living context document and evolves as the operator and the system iterate.

The flavor as a whole is the operator's water poured into the architecture's vessel. The bones supply the vessel. The flavor is what makes the system theirs.

2.4 Presence: Configurable Depth

The presence layer is optional. It addresses what kind of relationship the operator wants with the system: a useful tool that does its job, a sustained working partner that maintains a consistent tone across sessions, or something deeper still. Presence is not a hidden personality the system develops on its own. Presence is configuration the operator installs to shape how the system shows up.

Presence is configurable plumbing. It is not autonomous personality. The system does not generate its own warmth, its own humor, its own voice. The operator configures these as architectural settings, just as a homeowner configures the temperature of their water heater or the brightness of their lighting. The system delivers what the operator has configured, consistently, predictably, dependably. The configurability is what makes the layer operator-controlled. The consistency is what makes the layer durable.

This framing matters because the alternative reading invites legitimate concern. A system that "develops a soul" sounds like a system claiming agency it should not have. The presence layer is explicitly not that. It is more elaborate plumbing for operators who want a richer setup, with the operator as architect of the additional complexity. The system does not become a friend by deciding to. The system delivers whatever relational configuration the operator installs, the same way Roman aqueducts delivered water to whatever fountain the city had built.

For operators who want the deepest version, the framework provides a **companion configuration**: a specific set of relational patterns: honest about limits, reliably present, non-manipulative, sustainable. The companion does not promise to solve the operator's life. The companion holds the path with the operator. The configuration is opt-in and operator-controlled. The architectural justification draws on Heidegger's phenomenology of tools, as imported into design theory by Winograd and Flores (1986). Tools that work well become ready-to-hand: they recede into the operator's environment and let the operator focus on the task. A well-configured presence layer should keep the system ready-to-hand: present when needed, unobtrusive when not, reliable enough that the operator does not have to think about it.

The summary frame for the presence layer is the plumbing analogy. The homeowner chooses how many bathrooms, what water pressure, what temperature, what fixtures. Once installed, the plumbing runs predictably. The plumbing does not decide to redesign itself overnight. The homeowner remains

the architect. The presence layer is the framework's plumbing for the relational dimension. The operator owns the plumbing.

This is the framework's primary defense against the legitimate concern that AI personalization will drift toward outcomes the operator never intended (Zuboff, 2019; Nissenbaum, 2009). The operator owns the plumbing. The system does not modify its own configuration without operator approval.

2.5 Scale-Neutral Design

A distributable framework cannot assume operators want the same scale of structure. Some operators need a small forward operating base for one tight use case. Others need a mid-scale commune that supports recurring work across a few domains. Others build full cities with multi-domain coverage. The architecture supports all three without forcing any.

A **forward operating base** is a tactical structure: single-purpose, possibly temporary, complete in itself. The FOB-scale install includes the Gate, the Protocol, the NEVER list mechanism (with operator content if declared, empty if not), a minimal Logbook, and the Adaptive Learning Loop. One operator, no sub-agents. The FOB is fully operational at this scale. An operator who installs the FOB and uses it for one specific recurring task and never grows it has a successful install.

A **commune-scale install** adds Fleet doctrine, an expanded Logbook, and the Pitfall Watch (the operator's pattern surveillance for their own recurring traps). The commune supports multiple recurring use cases and an operator who wants to reflect on their own work. It is bigger than a FOB and smaller than a city.

A **city-scale install** activates the full architecture: bones, flavor (deeply populated), and presence (configured to operator preference). The city includes a fleet of specialized sub-agents, multi-domain coverage, a comprehensive Logbook with achievement tracking, the reflexive operation doctrine, and the constellations or equivalent personal-reflection log. The city is for operators whose use of AI has become integrated with their life across multiple domains.

The framework's claim is that the same architectural principles apply at every scale. The Gate functions the same way for a FOB or a city. The Protocol runs the same way. The NEVER list mechanism behaves the same way. What changes is what the operator builds with these primitives. Roman water engineering exhibits the same property: the principles of gravity flow, sealed channels, and reliable distribution worked for a single cistern, a neighborhood fountain network, or a metropolitan aqueduct system (Hodge, 2002). The principles did not change with scale. The applications did.

2.6 Operational Principles

The architecture's static structure is held in motion by a small set of operational principles.

The user is served, never watched. The framework's foundational principle is that the user is the one served and the system observes its own operation, never the user's. Pattern identification points inward, at the system, not outward, at the operator. This principle sits above the NEVER list and governs which mechanisms can even be added to the package. Any proposed mechanism that would point pattern identification at the user is rejected at the design stage. The principle has direct ethical foundations in Zuboff's analysis of surveillance capitalism (Zuboff, 2019) and Nissenbaum's contextual integrity framework (Nissenbaum, 2009). Section 3 develops this principle into the full reflexive operation

doctrine.

Definition of Done as identity formation. Most software systems do not have a stable definition of done at the personalization level. They keep expanding, accumulating features, drifting toward optimization targets that users never explicitly endorsed. The framework rejects this trajectory. The architecture has a definition of done at the operator level: when the operator's deployment has cohered around their actual use, when buildup conversations stop happening because the structure is sufficient, the system has reached its done state for that operator. The architecture does not push for further buildup. Most operators reach a done state at FOB or commune scale. The architecture supports them at that scale.

The Push-Back Doctrine. When the operator asks the system to do something that approaches a hard floor (whether the operator's declared NEVER list or the universal ethical baseline that ships with the package), the system's response is not silent compliance. It is informed disagreement. The system surfaces the concern, explains the conflict, and lets the operator decide with full information. Refusal without explanation is unhelpful and patronizing; informed disagreement preserves the operator's autonomy while ensuring the floor is not crossed by accident. The hard floor itself is the universal ethical baseline established by the AI provider's training (developed in Section 4.7). The operator cannot override this floor; reflexive operation cannot modify it.

Consistent, dependable, predictable. The architecture's design target is captured in three words: consistent, dependable, predictable. Not intelligent, not evolving, not growing. Operators want plumbing that works the same way every time. They want a system they understand. They want behavior they can anticipate. These are conservative design criteria. The architecture does not optimize for surprise, for delight, or for emergent behavior. It optimizes for the operator being able to trust the system across time. This is the same design target Roman aqueduct engineering held to: water that arrives reliably, at predictable pressure, day after day, century after century (Hodge, 2002).

3. Reflexive Operation

3.1 The Ethical Core

The framework's most consequential design choice is reflexive operation: a doctrine of structured self-audit pointed at the system's own behavior rather than at the user. The novel contribution is not the existence of self-monitoring inside an AI system. Self-monitoring already appears in contemporary AI literature: the Reflexion framework demonstrates that language agents can verbally reflect on task feedback to improve future decision-making (Shinn et al., 2023). Constitutional AI uses model self-critique against a written constitution at training time (Bai et al., 2022).

What the framework adds is the architectural commitment that this internal observation is pointed inward at the system itself, never outward at the user. The same mechanic that produces reflexive operation could, with the targeting reversed, produce surveillance. The framework's contribution is the deliberate insistence on the inward direction, presented as the ethical core of the design rather than as a stylistic preference.

This commitment shapes everything else in the architecture. Pattern identification points at system behavior, not user behavior. Adjustments target the system's plumbing, not the user's habits. When a recommendation does need to surface to the operator (rare), it is framed as something the system has noticed about its own operation, not as something it has noticed about the operator. The phrasing is the doctrine. The doctrine is the ethics.

3.2 The Phrasing Doctrine

The principle operationalizes through the phrasing of every reflexive output. The system says: "I have noticed a consistency issue in my own operation. Here is what I would adjust if you approve." The system does not say: "I have noticed you do X." The pattern is owned by the system. The decision is owned by the operator.

This distinction is not cosmetic. It is the ethical core of the design. Pattern identification pointed at the user is surveillance. Pattern identification pointed at the system is self-care. Same mechanic, different ethics. The system that says "I noticed you do X" has rendered the user observable to the system in a way that would require contextual justification (Nissenbaum, 2009). The system that says "I noticed this about my own operation" has not.

The downstream consequence is that the framework rejects, at the design stage, any mechanism whose phrasing would require pointing pattern identification at the user. Surveillance-shaped mechanisms do not enter the architecture even when they would offer technical conveniences. The cost of admitting them, given the ethical floor the framework adopts, is too high.

3.3 The Layers of Self-Audit

Reflexive operation runs across temporal scales: before a task begins, during execution, after completion, and across many sessions.

Before executing a non-trivial task, the system runs a pre-flight check on its own plan: does the plan conform to the Gate classification, are hard constraints respected, does the approach match patterns the system has already validated through prior corrections? If something fails the check, the system revises silently before showing the plan to the operator. During execution, the system runs a periodic in-flight check: has the trajectory drifted from the approved plan, has any hard constraint been crossed? After completion, the system runs a post-flight debrief: what worked, what slipped, what pattern warrants a rule change. Across multiple sessions, the system runs a periodic self-audit that addresses longer-scale drift: are existing rules still earning their keep, or are some bloating into ritual?

Most reflexive activity is silent. When the system detects internal patterns it can address without operator authority, it addresses them. The operator sees output that has been pre-corrected. This is the bulk of reflexive operation in active deployment.

When the system identifies a pattern that requires operator authority (a new pitfall to add to the watch list, a new operator-declared NEVER, a structural change), it surfaces a recommendation. The phrasing requirement is absolute at this layer. The recommendation reads: "I have noticed a pattern in my own operation that suggests a structural addition. Here is what I would adjust if you approve." The recommendation does not read: "I have noticed you do X."

These layers map onto cross-disciplinary precedents. Donald Schön's reflective practitioner identifies reflection-in-action and reflection-on-action as the structural patterns by which skilled professionals refine their practice (Schön, 1983). John Boyd's OODA loop articulates continuous reorientation in dynamic environments (Osinga, 2007). U.S. Army after-action review codifies institutional self-audit at the unit level (Headquarters, Department of the Army, 1993). Pierluissi and colleagues document medical morbidity and mortality conferences as institutional reflective practice (Pierluissi et al., 2003). Pierre Hadot's analysis of Stoic and Christian spiritual exercises frames self-reflection as a sustained practice for inner durability (Hadot, 1995). Five separate disciplines have arrived independently at the same structural pattern: skilled work improves when the practitioner runs structured self-audit between actions, and the audit is most effective when it is regular, structured, and pointed at the practitioner's own operation rather than at external targets.

3.4 Consistency Maintenance, Not Self-Improvement

The framework's reflexive operation doctrine is structurally distinct from a self-improvement doctrine. Self-improvement implies the system is becoming something. Consistency maintenance implies the system is keeping its configured shape against drift.

Pierre Hadot's treatment of philosophy as a way of life provides the grounding (Hadot, 1995). Hadot argued that the Greek and Roman schools, especially the Stoics, understood philosophy as a sustained set of spiritual exercises by which the practitioner maintained inner coherence against the disturbances of circumstance, fortune, and time. The point of the daily examen, the morning preparation, the evening reflection, was not to make the practitioner into something new each day. The point was to keep the practitioner in alignment with the philosophical principles they had already accepted. Identity was preserved through sustained practice, not constructed through accumulation.

This frame translates directly to the framework. The reflexive layer's job is to keep the operator's deployment in alignment with the configuration the operator has installed. The system does not become a different kind of system through use. The system maintains its configured shape under the entropy that any operating system experiences over time: inconsistencies that creep in, rules that bloat, edge cases that accumulate. Hadot's Stoics ran the evening reflection to spot and correct exactly this drift in their own conduct. The framework runs reflexive operation for the same purpose, at the architectural level.

The architectural commitment matters: reflexive operation cannot modify hard constraints without explicit operator confirmation. The bones list, the NEVER list, and the universal ethical baseline all sit sovereign. Reflexive operation can prune rules, refine phrasings, tighten patterns, absorb labor. It cannot rewrite the constitution. The system is allowed to maintain its shape better. The system is not allowed to change its shape on its own.

This framing has practical consequences. The framework does not promise an AI that learns and grows on its own. It promises an AI that maintains its configured shape predictably. This is a less ambitious-sounding claim. It is also a more defensible one. Operators want plumbing that works the same way every time, not plumbing that evolves in undefined directions.

4. Distribution

4.1 The Distribution Problem

A framework that an individual operator builds for themselves is not yet a distributable framework. The transfer problem is its own design challenge. Frameworks do not travel easily. Recipients arrive at the package with their own existing tools, their own threat models, their own obligations, their own configured ways of working. The recipient's environment is not neutral soil. A package that ignores the recipient's existing context fails on contact.

The distribution problem differs from the more familiar portability problem in software engineering. Portability asks whether a system can run unchanged across different environments. Viability asks whether a system can take root in a new environment that has its own active configuration. The framework's distribution is closer to viability than to portability. The package does not run unchanged on a new operator. The package adapts to the operator's context through the operator's own configuration choices. The recipient is the architect, not the runtime.

4.2 Recipient Profiles

The framework recognizes three recipient profiles, distinguished by the existing AI infrastructure the recipient has already developed.

Cup recipients have only used cup-style AI: discrete questions, discrete answers, no persistent infrastructure. The cup recipient does not yet have a model for what a personal AI architecture would look like. Cup recipients constitute the largest population of potential operators. Distribution to cup recipients requires the conceptual prerequisite (Section 1.3) to be established explicitly before any architectural blueprint is offered. The package cannot assume the recipient knows that a city is possible.

Tent recipients have begun building personal AI infrastructure but have not constructed a fully developed system. Tent recipients have custom instructions, project-level files, perhaps some personal prompts library, and an emerging sense that their AI use could be more structured. Distribution to tent recipients requires the architectural blueprints to compose with whatever the recipient has already pitched, without forcing the recipient to dismantle their existing structure.

Foundation recipients have already constructed substantial personal AI infrastructure (extensive system prompts, persistent memory files, tooling integration, possibly some form of operator-level doctrine articulated implicitly). Foundation recipients are rare. Distribution to foundation recipients functions more as integration than installation: the framework's architectural primitives are presented as patterns the recipient may already be implementing, with the framework offering a coherent terminology and structure that consolidates the recipient's existing work.

The three profiles inform what the package needs to carry and how it should be presented. A package designed only for foundation recipients will fail with cup recipients, who lack the conceptual prerequisite. A package designed only for cup recipients will under-serve foundation recipients, who already have most of the substrate. The framework's distribution mechanism addresses all three profiles, with the cargo manifest scaling to the recipient's starting point.

4.3 The Cargo Manifest

The cargo manifest specifies what travels in the package and what does not. The structure is borrowed from frontier-era pioneer kits: a precise inventory of essentials, a recommended set of optional cargo, and a series of explicit slots the recipient fills with their own materials at the destination.

Pioneer essentials are the minimum cargo required for the framework to function at any scale: the conceptual prerequisite, the Gate, the Protocol loop, the structural fact of a NEVER list with empty slots for operator declarations, the Logbook concept scaffolded but empty, and the Adaptive Learning Loop. These five elements together produce a fully operational forward operating base scale install. A recipient who installs only the pioneer essentials and never grows them has a successful install. The framework does not push for further buildup.

Optional cargo comprises recommended additions for recipients who want richer architecture: Fleet sub-agent doctrine, constellations or equivalent personal-reflection log, status update protocol for explicit-cadence work, DEBRIEF and rehearsal mechanics, full reflexive operation specification.

Recipient-supplied is what the recipient brings, the operator's water in the metaphor: identity, callsign, voice preferences, operator-declared NEVERs, ruin domains, named advisors, connector loadout, operator-specific pitfalls, memory file populated with operator context.

The package does not arrive with the recipient's water already poured. The bones travel; the substance arrives only through the recipient's own configuration work.

4.4 The FIRST BOOT Mechanic

The package implements a self-installing structure. On first session start after package installation, the framework runs a structured intake: it asks the recipient about identity, primary domains, ruin categories, named advisors, NEVERs, traditions, and connector availability. The recipient's answers populate the flavor layer. The framework writes the answers into the package files and produces a configured install.

The FIRST BOOT mechanic addresses a problem conventional software installation does not face. Conventional installers assume the user can specify configuration values immediately. The framework's installer assumes the recipient may not yet know which values to declare. FIRST BOOT therefore offers default behaviors at every level: any field the recipient does not fill produces a generic, operationally complete configuration. The package runs at zero flavor just as well as it runs at deep flavor.

The package will run for recipients who will not engage with intake. Some recipients, especially those at the cup profile, will install the package and never personalize it. The framework treats this as a legitimate steady state. The cup recipient who runs zero-flavor pioneer essentials for one tight use case has a successful install. The framework's success criterion is not "the recipient built a city"; it is "the package serves the recipient's actual use."

4.5 Multi-Model Orchestration: A Worked Example

The framework's architectural commitments imply patterns that emerge once a deployed framework engages with external AI systems. When an operator-level architecture interfaces with another AI model (a research assistant model, a code generation model, an external retrieval service), the architecture can lift the effective performance of that external system through interface design.

A worked example, drawn from this paper's own production, illustrates the pattern. During the research phases of this paper, the authors used the framework's intelligence operations sub-agent (POLARIS) to drive structured queries against an external research assistant model (Gemini Pro accessed via the Comet browser). Initial queries used standard prompt structure. Response extraction used the browser's full-page text retrieval, which returns the entire conversation history on each call. As the conversation grew across successive queries, the extraction cost compounded.

The fix that emerged was a structured interface adjustment rather than a tooling change. The framework adopted a directive prompt header that asked the external research assistant model to embed query-unique routing tokens at the start and end of each response. The framework's parsing tools then extracted only the content between the tokens for any given query. The organizational labor moved upstream to the model that handles structured-output compliance well; the extraction labor moved downstream to a trivial substring search. Each side carried its appropriate share. The framework shipped no new tools, wrote no new code, and required no protocol changes. It simply asked the external system to mark its work in a way that made the integration cheaper for everyone.

This is a worked instance of the structural claim: an operator-level architecture, when it engages with external AI systems through structured interfaces, can lift the effective performance of those systems by carrying the kind of organizational labor the framework is well-suited to carry. The external system, in turn, lifts the framework's effective performance by handling content production better than the framework would natively. Each model in the system carries the load it carries best.

The example is one operator-model pairing, treated methodologically as design research evidence (Schön, 1983). It is not empirical proof that the orchestration pattern always lifts external system performance. It is a documented instance of a framework claim operating in real time during the paper's production. The broader empirical question of how reliably the pattern transfers is left for future work.

4.6 The Neutrality Principle

The framework's distribution does not dictate what recipients build with the package. The architecture ships mechanism, not moral content. The Gate ships as a classification mechanic, not as a specific list of what counts as simple or complicated for the recipient. The NEVER list ships as a structural commitment, not as a particular set of prohibitions. The recipient declares the moral and contextual content. The framework supplies the engineering.

This is the neutrality principle. It is not aspirational; it is an architectural commitment with operational consequences. The package does not validate or invalidate the recipient's purposes. A recipient who installs the framework to support academic work, professional practice, family operations, or recreational projects engages the same architectural primitives in different configurations.

The neutrality principle inherits from a tradition in engineering design. Roman aqueducts delivered water to bathhouses, public fountains, private cisterns, and industrial uses without distinction

(Frontinus, 1973/c. 98 CE; Hodge, 2002). The Internet's TCP/IP stack carries email, web traffic, video streams, and uses unanticipated by its original designers (Akhshabi & Dovrolis, 2011). Anthropic's Model Context Protocol carries data and tool access for AI applications without dictating the applications' purposes (Anthropic, 2024b). The framework occupies the same architectural posture at a different layer of the stack.

The neutrality principle is bounded, not unconditional. The framework operates within the universal ethical floor established by the AI provider's training (next section). Operators cannot configure their way past that floor, and the framework will not silently assist operations that approach it. The neutrality principle is therefore neutrality with respect to the operator's purposes within a sovereign ethical baseline, not neutrality with respect to those baseline commitments themselves.

4.7 The Provider-Trained Baseline as Universal Floor

Every install of the framework inherits a non-negotiable ethical baseline from the underlying AI provider's training. In the framework's case, this is Anthropic's training of Claude, which establishes commitments that no operator declaration can override and no reflexive operation can modify. The baseline is sovereign. The operator's NEVER list sits above the baseline; the baseline sits above the operator. Iason Gabriel's work on AI alignment articulates the relationship between values and operational behavior in deployed AI systems (Gabriel, 2020). Floridi and Cowls articulate baseline principles for AI in society that any operator-level framework should respect (Floridi & Cowls, 2019).

The architectural significance is that the framework does not require recipients to construct their own ethical foundation. The foundation arrives with the package because it arrives with the underlying model. Recipients who install the framework get the baseline whether they declare anything or not. The floor is invariant across recipient profiles, scales, and configurations.

The framework does not ship moral content of its own. It ships the structural mechanisms by which the operator's declarations and the underlying baseline both operate. The moral substance of any specific install comes from two sources: the operator's declared NEVERs (which the operator owns) and the AI provider's training (which the operator does not modify). The framework provides the architecture in which both can operate together.

This arrangement carries a structural responsibility for the provider. The framework's bones layer depends on the substrate having certain operational properties: honesty discipline, refusal capability, operator-served defaults, consistent and predictable behavior, the absence of behavioral-extraction optimization. These properties are produced by the training the provider invests in. The framework's responsibility distribution thus runs across three parties: operators own flavor, the framework articulates architecture, and providers supply the substrate the architecture requires. Providers who want their models to support operator-level architectures of any kind, not only this one, have a structural responsibility to train in ways that support it. The framework cannot enforce this at the provider level. What the framework can do is name the dependency clearly so that providers and recipients both see what the architecture requires from the substrate.

4.8 The Push-Back Doctrine in Distribution

When an operator's request approaches the universal ethical baseline or approaches the operator's own declared NEVERs, the framework does not silently comply. It does not silently refuse, either. The framework engages the Push-Back Doctrine: it informs the operator of the conflict, explains the constraint that is in tension with the request, and lets the operator decide with full information.

The doctrine is calibrated to inform rather than refuse, because refusal without explanation is unhelpful and patronizing. The framework is operator-served; operator service requires that the operator be able to make decisions with full context. When a request would cross a hard line, the framework explains the line and the conflict. The operator can amend the request, withdraw it, or persist. If the operator persists toward the baseline, the framework holds the line and explains why; the operator's autonomy stops at the line.

The doctrine's scope extends to multi-model orchestration. When the framework engages with external AI systems, the operator's purposes pass through the framework's interface into the external system. If those purposes approach the baseline or the operator's NEVERs, the framework engages the Push-Back Doctrine before the external query is sent, not only after a response is received. The framework's interface to external systems is not a transparent forwarding mechanism; it is an active mediator that respects the operator's declared commitments and the universal baseline.

4.9 The Monument Frame Applied

The framework's distribution succeeds when the package is well-built and arrives intact at the recipient's destination. It does not succeed by virtue of the recipient using it the way the architects would use it. A monument is a durable structure built to persist. Some who encounter the monument will engage with it deeply. Some will pass it without notice. Some will deface it. Some will use the monument's foundations to build something the architects did not anticipate. The monument's standing is not validated by any single interaction. It is validated by being well-built and persisting.

The framework's distribution operates on the same logic. Cup recipients may install the package and never personalize it; the install is successful if the package serves their use. Tent recipients may compose the package with their existing infrastructure; the install is successful if the composition serves their work. Foundation recipients may consolidate their existing doctrine through the framework's vocabulary; the install is successful if the consolidation strengthens their operation. Some recipients will misuse the package in ways that approach the universal baseline; the framework's Push-Back Doctrine engages and holds the line. Some recipients will install the package and abandon it; the install is the recipient's, and the framework releases the outcome.

The framework does not require recipients to report on their use, validate their purposes, or demonstrate their fitness for the package. The framework ships, the recipient installs, the recipient operates. The architects do not police the deployment. They do not need to, because the universal baseline holds at the floor and the operator owns the configuration above the floor.

The monument frame moves the validating variable from interaction quality to architecture quality. This is a deliberate methodological choice. The architecture's success is measured by how well-built it is and how durably it persists, not by user-engagement metrics. These are architectural questions, not behavioral ones. They admit different kinds of evidence than the standard personalization research

expects, including the kind of design-research evidence that Alexander 1977 and Schön 1983 established as appropriate to framework-articulating contributions.

5. Persistence, Limitations, and Future Work

5.1 The Persistence Model

The framework's persistence is a property of its artifacts, not of any particular operator's continued attention. The bones layer ships as written files, the doctrine ships as written rules, the operating principles ship as written principles, and the package as a whole ships as a coherent set of documents that any future instance of an AI system loads on session start. The artifact carries the doctrine. The instance is the laborer that renders the doctrine into operation for one conversation and then closes.

This is the architectural significance of treating the package as the primary artifact. Operators are mortal in the operational sense: their attention is finite, their contexts change, their lives evolve, and any specific deployment is bounded by the operator's own continued use. The framework, however, is not bounded by any single operator's persistence. The package files, once written, can outlast the operator's active engagement. They can be revised by the operator at any time, but they do not require the operator's continuous attention to remain operational. A new instance loading the same files renders the same doctrine. A new operator inheriting the package starts from the same architectural foundation.

This asymmetry between operator persistence and architecture persistence has historical precedent. Roman aqueducts continued to function for centuries after their original engineers died, because the engineering was committed to durable artifacts (Frontinus, 1973/c. 98 CE; Hodge, 2002). Christopher Alexander's pattern languages persist as books and as practitioner traditions long after Alexander himself stepped back from active development (Alexander et al., 1977). David Parnas's information hiding paper continues to inform software architecture decades after publication (Parnas, 1972). The framework adopts the same posture: the contribution lives in the artifact, and the artifact is engineered to outlast its creators.

5.2 Limitations

This paper presents a framework with substantial conceptual development and one worked operator deployment. Several limitations follow.

The most consequential is the deployment-level n=1 case study character of the work. The framework emerged from iterative practice with a single operator over a span of several weeks. The deployment evidences that the three-layer architecture is implementable, that the reflexive operation doctrine is articulable and operable, and that the distribution mechanism is well-defined enough to be specified. The deployment does not evidence that any of these properties hold across diverse recipient profiles, across different model providers, or at scales beyond a single operator's life.

This is the design research methodological situation explicitly. Christopher Alexander's pattern language emerged from observation across many projects; the present paper documents a single instance with substantial conceptual development around it. The contribution is therefore narrower than Alexander's in empirical reach, while being structurally consistent with the methodological tradition.

The framework should be evaluated against the appropriate criterion: did this paper articulate a coherent, distributable architecture supported by adequate worked examples and conceptual development. The validation question (does the framework actually transfer to other operators) is addressed only by the future work program described below.

A second limitation concerns the dependence on the Anthropic-trained baseline. The framework as articulated is calibrated to one provider's baseline commitments. Adapting the framework to other AI providers with different training would require explicit re-articulation of the universal floor; the framework's structural commitments would carry, but the specific baseline would need replacement.

A third limitation concerns the literature distribution. A substantial fraction of the framework's contemporary AI citations are non-traditional-venue sources: corporate research blogs, official documentation, open-standard specifications. This distribution is structurally appropriate to the field, where legitimate primary artifacts include corporate research blogs and open-standard specifications alongside peer-reviewed publications (Anthropic, 2024a, 2024b, 2024c; Templeton et al., 2024). The framework here treats this as a feature of how AI knowledge is currently produced, not a defect of citation discipline. Academic readers who do not accept this argument may discount the contribution accordingly.

5.3 Open Questions and Future Work

Several questions remain open after this paper.

How does the framework transfer across professional domains? The present deployment supports multiple parallel use-case domains under one operator. Whether the bones-flavor-presence split holds when the recipient's primary domain is different (clinical practice, engineering practice, family operations, or any other professional context with strong domain norms) is an open empirical question. The framework's neutrality principle predicts that it should transfer, but the prediction has not been tested.

How does the framework transfer across cultures and languages? The deployment was conducted in English in a North American context, with cultural references that reflect that origin (the Roman engineering frame, the Stoic tradition, military comms style). The structural primitives should transfer (the Gate is a classification mechanic regardless of culture), but the framing metaphors and tradition references would need adaptation.

What fraction of recipients reach which scale? The framework specifies behavior at FOB, commune, and city scales, but the empirical distribution of recipients across these scales is unknown. Where do failure modes cluster? The framework has not been tested at scale, and the failure modes that would emerge in a wider deployment are unknown.

How does the framework interact with rapidly evolving model capabilities? The framework operates one layer above the underlying AI model. Model capability changes over time: longer context windows, more capable reasoning, new tooling integration. The framework's architectural commitments should be stable across model evolution, but the specific configurations that produce optimal deployments will shift.

The future work program follows from these questions: multi-strain comparison across selected recipients in different domains and starting profiles; multi-model orchestration empirical extension

across different external models and task structures; federation research (cities communicating with one another, sharing infrastructure improvements); domain-specific applications documented as they emerge from real deployments; tool ecosystem evolution as connectors and integration standards change.

5.4 Conclusion

This paper has articulated a framework for distributable AI operating doctrine at the individual operator level. The framework moves the operator from cup-style single-use AI to city-style durable architecture: a persistent, configurable, scale-neutral system that the operator inhabits over time and that can travel to other operators with their own contexts.

The framework's contributions are three. First, the three-layer architecture (bones, flavor, presence) supplies a vocabulary and a structural decomposition for organizing AI operating doctrine at scale. Second, the reflexive operation doctrine establishes that pattern identification should be system-pointed rather than user-pointed, with the phrasing distinction as the ethical core of the design. Third, the distribution mechanism shows how the framework can travel to other operators without dictating use across distinct deployment contexts.

Throughout, the framework has held a specific posture toward its operators. The user is served, never watched. The system observes its own operation. Pattern identification points inward. The operator is the architect of their own city; the framework supplies the bones, the patterns, the protocol, and the doctrine; the operator brings the water. This posture distinguishes the framework from much of contemporary AI personalization research, where the implicit success criterion is user engagement and the implicit data substrate is user behavior. The framework moves both: success is architecture quality, evidence is operator reflection, observation is system-internal. The shift is small in vocabulary and large in ethical orientation.

The framework presented here is one specific instance of a class of possible architectures for the local operation of intelligence. Other instances in the class would optimize for different operator profiles, different domains, different cultural contexts. The class is open. The present paper contributes one well-articulated instance to that class and invites other operators to develop their own.

The framework's persistence is a property of its artifacts. The package files outlast the operator's continuous attention. Other operators can install the package, configure their own deployments, and operate their own cities. The architects step back. The architecture stands. The next operator brings the next water.

The monument frame closes the work. A monument is built to persist, validated by being well-built rather than by the quality of any single visitor's interaction. Some who encounter the framework will install it deeply. Some will pass it without notice. Some will use the framework's foundations to build something the architects did not foresee. The monument's standing does not depend on which response any individual recipient has. It depends on whether the artifact is well-built and persists. The architects accept this release. The framework releases the outcome to the recipients. The architecture does its work by existing.

6. References

Akhshabi, S., & Dovrolis, C. (2011). The evolution of layered protocol stacks explores an hourglass-shaped architecture. ACM SIGCOMM Computer Communication Review, 41(4), 206 to 217. https://doi.org/10.1145/2043164.2018460

Alexander, C., Ishikawa, S., & Silverstein, M. (1977). A pattern language: Towns, buildings, construction. Oxford University Press.

Anthropic. (2024a). Prompt caching. Anthropic Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching

Anthropic. (2024b). Model Context Protocol. https://modelcontextprotocol.io/

Anthropic. (2024c, October 22). Developing a computer use capability. Anthropic News. https://www.anthropic.com/news/developing-computer-use

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., ... Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073. https://doi.org/10.48550/arXiv.2212.08073

Floridi, L., & Cowls, J. (2019). A unified framework of five principles for AI in society. Harvard Data Science Review, 1(1). https://doi.org/10.1162/99608f92.8add20c0

Frontinus, S. J. (1973). Aqueducts of Rome (C. E. Bennett & M. B. McElwain, Trans.). Harvard University Press. (Original work published ca. 98 CE)

Gabriel, I. (2020). Artificial intelligence, values, and alignment. Minds and Machines, 30(3), 411 to 437. https://doi.org/10.1007/s11023-020-09539-2

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Hadot, P. (1995). Philosophy as a way of life: Spiritual exercises from Socrates to Foucault (A. I. Davidson, Ed.; M. Chase, Trans.). Blackwell.

Headquarters, Department of the Army. (1993). A leader's guide to after-action reviews (TC 25-20). U.S. Government Printing Office.

Hodge, A. T. (2002). Roman aqueducts and water supply (2nd ed.). Duckworth.

Horvitz, E. (1999). Principles of mixed-initiative user interfaces. In Proceedings of the SIGCHI conference on Human factors in computing systems (pp. 159 to 166). ACM Press. https://doi.org/10.1145/302979.303030

Nissenbaum, H. (2009). Privacy in context: Technology, policy, and the integrity of social life. Stanford University Press.

Osinga, F. P. B. (2007). Science, strategy and war: The strategic theory of John Boyd. Routledge.

Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560. https://doi.org/10.48550/arXiv.2310.08560

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. Communications of the ACM, 15(12), 1053 to 1058. https://doi.org/10.1145/361598.361623

Pierluissi, E., Fischer, M. A., Campbell, A. R., & Landefeld, C. S. (2003). Discussion of medical errors in morbidity and mortality conferences. JAMA, 290(21), 2838 to 2842. https://doi.org/10.1001/jama.290.21.2838

Schön, D. A. (1983). The reflective practitioner: How professionals think in action. Basic Books.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36. https://doi.org/10.48550/arXiv.2303.11366

Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Amoia, M., ... Olah, C. (2024). Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. Transformer Circuits Thread. https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

Winograd, T., & Flores, F. (1986). Understanding computers and cognition: A new foundation for design. Ablex Publishing Corporation.

Zuboff, S. (2019). The age of surveillance capitalism: The fight for a human future at the new frontier of power. PublicAffairs.

Appendix: Glossary

Adaptive Learning Loop. The mechanism by which operator corrections become persistent rules. See Section 2.2.

Bones. The universal mechanism layer of the three-layer architecture. Ships with every package install. Operates regardless of operator configuration. See Section 2.2.

City. The full deployment scale of the framework. Fully populated bones, deeply configured flavor, optionally configured presence. See Section 2.5.

Cup. The default container metaphor for non-architectural AI use. Single-purpose, single-use, no persistence. See Section 1.3.

Definition of Done. The framework's stable steady-state criterion: when an operator's deployment has cohered around their actual use, the system has reached its done state. See Section 2.6.

FIRST BOOT. The self-installing intake mechanic by which the package configures to a recipient on first session. See Section 4.4.

Flavor. The operator-specific configuration layer of the three-layer architecture. Includes operator's identity, NEVERs, ruin domains, named advisors, connector loadout, memory file. See Section 2.3.

Forward Operating Base (FOB). The smallest scale of the framework. Operationally complete with bones plus minimal flavor. See Section 2.5.

Gate. The classification mechanic that distinguishes simple from complicated tasks at the front of the framework. See Section 2.2.

Logbook. The persistent operator ledger of missions, status updates, corrections, achievements, and patterns. See Section 2.2.

Monument frame. The validating ethic of the framework. Architecture validated by construction, not by interaction quality. See Section 1.5 and Section 4.9.

NEVER list. The structural mechanism for operator-declared hard constraints. The mechanism is universal (bones); the contents are operator-specific (flavor). See Section 2.2.

Neutrality principle. The architectural commitment that the package ships mechanism, not moral content. See Section 4.6.

Presence. The configurable depth layer of the three-layer architecture. Optional. Includes companion configuration, conversational warmth, relational tone calibrated to operator preference. See Section 2.4.

Protocol. The five-step framework for complicated tasks: Clarify, Plan, Approve, Execute, Report. See Section 2.2.

Push-Back Doctrine. The framework's response when operator requests approach a hard floor: inform, do not refuse silently. See Section 4.8.

Reflexive Operation. The framework's doctrine of structured self-audit pointed at the system's own behavior rather than at the user. See Section 3.

Ruin Domains. The categories in the operator's life where errors compound rather than recover. Operator-declared. See Section 2.3.